

Redis & Celery

The Complete Beginner-to-Advanced Guide

Installation • Core Concepts • Hands-on Usage • Real Project Patterns • Production Tips

Table of Contents

Part 1: Redis

1. What is Redis and why use it
2. Installing Redis (Linux, macOS, Windows, Docker)
3. Redis CLI basics
4. Redis data types with examples
5. Redis with Python (redis-py)
6. Common Redis use cases
7. Redis configuration and persistence essentials

Part 2: Celery

8. What is Celery and why use it
9. Installing Celery
10. Your first Celery task
11. Calling tasks: delay, apply_async, chains, groups
12. Celery worker, concurrency, and Celery Beat (scheduling)
13. Monitoring with Flower
14. Production best practices and troubleshooting
15. Putting it together: Django/Flask + Redis + Celery example

Part 1 — Redis

1. What is Redis and why use it

Redis (REmote DIctionary Server) is an in-memory, key-value data store. Because it keeps data in RAM, reads and writes are extremely fast (usually sub-millisecond). It is used as a cache, a message broker, a session store, a rate limiter, a leaderboard engine, and a real-time analytics store. Redis also supports optional persistence to disk, so data is not necessarily lost on restart.

Why people learn Redis:

- Speed: in-memory access is far faster than a traditional disk-based database.
- Simplicity: simple commands like SET, GET, EXPIRE, LPUSH.
- Versatility: strings, lists, sets, sorted sets, hashes, streams, pub/sub.
- It is the most common broker/backend used with Celery for background task processing.

2. Installing Redis

Option A — Docker (recommended for beginners, works on any OS)

```
# pull and run redis in the background, exposing default port 6379
docker run -d --name redis-server -p 6379:6379 redis:7-alpine

# verify it's running
docker exec -it redis-server redis-cli ping

# should reply: PONG
```

Option B — Ubuntu / Debian Linux

```
sudo apt update
sudo apt install redis-server -y

# start the service
sudo systemctl enable redis-server
sudo systemctl start redis-server

# check status
sudo systemctl status redis-server

# test it
redis-cli ping # should return PONG
```

Option C — macOS (using Homebrew)

```
brew install redis

# start now and on login
brew services start redis

# test it
redis-cli ping    # should return PONG
```

Option D — Windows

Redis does not officially support Windows. The two practical options are:

- Use Docker Desktop and run the Docker command from Option A (recommended).
- Use WSL2 (Windows Subsystem for Linux) and follow the Ubuntu instructions inside it.

Tip: whichever method you use, the goal is the same — a Redis server listening on **localhost:6379**. Once `redis-cli ping` returns PONG, Redis is installed correctly and you can move on.

3. Redis CLI basics

Open the interactive shell with `redis-cli` and try these commands:

```
$ redis-cli

127.0.0.1:6379> SET name "John"
OK
127.0.0.1:6379> GET name
"John"
127.0.0.1:6379> EXISTS name
(integer) 1
127.0.0.1:6379> EXPIRE name 60
(integer) 1
127.0.0.1:6379> TTL name
(integer) 57
127.0.0.1:6379> DEL name
(integer) 1
127.0.0.1:6379> KEYS *
(empty array)
```

4. Redis data types with examples

Strings

```
SET counter 10
INCR counter      # 11
INCRBY counter 5  # 16
APPEND counter "x" # value becomes "16x"
```

Lists (ordered, good for queues)

```
LPUSH tasks "task1"
LPUSH tasks "task2"
LRANGE tasks 0 -1      # ["task2", "task1"]
RPOP tasks              # removes from the right end
```

Sets (unique, unordered)

```
SADD tags "python" "redis" "celery"
SMEMBERS tags
SISMEMBER tags "redis"    # 1 (true)
```

Sorted Sets (great for leaderboards / rankings)

```
ZADD leaderboard 100 "alice"
ZADD leaderboard 250 "bob"
ZRANGE leaderboard 0 -1 WITHSCORES
```

Hashes (like a mini object/dictionary)

```
HSET user:1 name "Alice" age 30
HGET user:1 name
HGETALL user:1
```

5. Redis with Python (redis-py)

```
pip install redis
```

```
import redis

r = redis.Redis(host="localhost", port=6379, db=0, decode_responses=True)

r.set("greeting", "hello world")
print(r.get("greeting"))          # hello world

r.hset("user:1", mapping={"name": "Alice", "age": "30"})
print(r.hgetall("user:1"))        # {'name': 'Alice', 'age': '30'}

r.expire("greeting", 30)          # key auto-deletes after 30 seconds
```

6. Common Redis use cases

- **Caching:** store expensive database query results with an expiry (TTL) to reduce load.
- **Session storage:** store logged-in user sessions for fast lookup across servers.
- **Rate limiting:** use INCR + EXPIRE to count requests per user per minute.
- **Pub/Sub messaging:** real-time notifications between services.
- **Message broker:** Redis is the most common backend used by Celery to queue background jobs (this is why you are learning both together).
- **Leaderboards / counters:** sorted sets for game scores, view counts, trending items.

7. Configuration and persistence essentials

By default Redis config lives at `/etc/redis/redis.conf` on Linux. Two persistence modes matter for beginners:

- **RDB (snapshotting)**: periodic point-in-time dumps of the dataset to disk. Fast, but you can lose the last few minutes of data on a crash.
- **AOF (append only file)**: logs every write operation, so recovery is more complete but the file grows larger and rewrites are slower.

For a task-queue / cache use case, the defaults are fine to start. You can check memory usage and general stats with `redis-cli INFO`.

Part 2 — Celery

8. What is Celery and why use it

Celery is a distributed task queue for Python. It lets you run code asynchronously, in the background, outside your main web request/response cycle. Typical uses: sending emails, processing images/videos, generating PDFs or reports, calling slow third-party APIs, running scheduled jobs (like a cron replacement).

How the pieces fit together:

- **Producer:** your app (e.g. Django/Flask view) that says 'run this task later'.
- **Broker:** a message queue that holds pending tasks — this is where Redis comes in.
- **Worker:** a separate Celery process that picks up tasks from the broker and executes them.
- **Result backend** (optional): stores the return value/status of a task — also often Redis.
- **Celery Beat** (optional): a scheduler that periodically enqueues tasks (like cron).

Mental model: your web app doesn't call the function directly. It drops a message describing the task into Redis, and a separate Celery worker process picks it up and runs it whenever it's free. This keeps your web app fast and responsive.

9. Installing Celery

Celery requires Python already installed, and needs a running broker — we'll use the Redis you set up in Part 1.

```
# create and activate a virtual environment (recommended)
python -m venv venv
source venv/bin/activate          # on Windows: venv\Scripts\activate

# install celery with redis support
pip install celery redis
```

10. Your first Celery task

Project layout:

```
myproject/
  celery_app.py
  tasks.py
```

celery_app.py — configure Celery to use Redis as the broker and result backend:

```
# celery_app.py
from celery import Celery

app = Celery(
    "myproject",
    broker="redis://localhost:6379/0",
    backend="redis://localhost:6379/1",
)

app.conf.update(
    task_serializer="json",
    result_serializer="json",
    accept_content=["json"],
    timezone="UTC",
    enable_utc=True,
)
```

tasks.py — define a task:

```
# tasks.py
from celery_app import app
import time

@app.task
def add(x, y):
    return x + y

@app.task
def slow_task(seconds):
    time.sleep(seconds)
    return f"Slept for {seconds} seconds"
```

Start the worker in a terminal (keep it running):

```
celery -A tasks worker --loglevel=info

# on Windows, add the solo pool:
celery -A tasks worker --loglevel=info --pool=solo
```

In a separate Python shell, call the task:

```
from tasks import add

result = add.delay(4, 6)      # sends the task to Redis, returns immediately
print(result.id)             # unique task id
print(result.status)         # PENDING / STARTED / SUCCESS / FAILURE
print(result.get(timeout=10)) # blocks until result is ready -> 10
```

11. Calling tasks: delay, apply_async, chains, groups

delay() vs apply_async()


```
add.delay(2, 3) # shortcut

add.apply_async(
    args=(2, 3),
    countdown=10,      # run 10 seconds from now
    expires=60,        # discard if not run within 60 seconds
    retry=True,
)
```

Chaining tasks (run one after another, passing results)

```
from celery import chain

# add(2,3) -> result fed into add(result, 10)
workflow = chain(add.s(2, 3), add.s(10))
result = workflow.apply_async()
print(result.get()) # 15
```

Groups (run tasks in parallel)

```
from celery import group

job = group(add.s(i, i) for i in range(5))
result = job.apply_async()
print(result.get()) # [0, 2, 4, 6, 8]
```

Automatic retries on failure

```
@app.task(bind=True, max_retries=3, default_retry_delay=5)
def fetch_url(self, url):
    import requests
    try:
        return requests.get(url, timeout=5).text
    except requests.RequestException as exc:
        raise self.retry(exc=exc)
```

12. Celery worker, concurrency, and Celery Beat (scheduling)

Concurrency: how many tasks run at once

```
# run 4 worker processes in parallel
celery -A tasks worker --loglevel=info --concurrency=4
```

Celery Beat — running tasks on a schedule (like cron)

```
# celery_app.py (add this)
from celery.schedules import crontab

app.conf.beat_schedule = {
    "add-every-30-seconds": {
        "task": "tasks.add",
        "schedule": 30.0,
        "args": (16, 16),
    },
    "daily-report-at-7am": {
        "task": "tasks.generate_daily_report",
        "schedule": crontab(hour=7, minute=0),
    },
}
```

Run beat in its own terminal, alongside the worker:

```
celery -A tasks beat --loglevel=info
```

13. Monitoring with Flower

Flower is a web dashboard for Celery: see active workers, task history, success/failure rates, and even revoke tasks from the browser.

```
pip install flower

celery -A tasks flower --port=5555
# then open http://localhost:5555 in your browser
```

14. Production best practices and troubleshooting

- Use separate Redis logical databases for the broker and result backend (e.g. /0 and /1) so they don't collide.
- Always set `time_limit` / `soft_time_limit` on long-running tasks so a stuck task can't block a worker forever.
- Use `acks_late=True` for critical tasks so they are re-queued if a worker crashes mid-task.
- Run Celery as a background service with a process manager (systemd, supervisor, or Docker) instead of leaving a terminal open.
- Keep tasks small and idempotent (safe to run twice) — retries and worker restarts can re-run tasks.
- Never pass large or non-serializable objects (like Django model instances) into a task; pass IDs and re-fetch inside the task.
- If tasks aren't running: confirm Redis is reachable (`redis-cli ping`), confirm the worker terminal shows the task registered, and check that broker URLs match exactly between the app and the worker.

15. Putting it together: a minimal Flask + Redis + Celery example

```
# app.py
from flask import Flask, jsonify
from tasks import add

app = Flask(__name__)

@app.route("/add/<int:x>/<int:y>")
def trigger_add(x, y):
    task = add.delay(x, y)
    return jsonify({"task_id": task.id})

@app.route("/result/<task_id>")
def get_result(task_id):
    from celery_app import app as celery_app
    result = celery_app.AsyncResult(task_id)
    return jsonify({"status": result.status, "result": result.result})

if __name__ == "__main__":
    app.run(debug=True)
```

To run the whole stack, you need three terminals running at once:

```
# Terminal 1: Redis (if not already running via Docker/service)
redis-server

# Terminal 2: Celery worker
celery -A tasks worker --loglevel=info

# Terminal 3: Flask app
python app.py
```

Suggested learning path: (1) get Redis running and comfortable with the CLI, (2) get a single Celery task running end-to-end, (3) add chains/groups and retries, (4) add Celery Beat for scheduled jobs, (5) add Flower for visibility, (6) wire it into a real Flask/Django app and deploy with a process manager.

End of guide.